

Übungsklausur in Programmieren und Software Engineering

Klassen: 6xAIF, 6xKIF, 8ABIF, 8ACIF

Arbeitszeit: 300 Minuten

Generelle Hinweise zur Bearbeitung

Sie haben 5 Stunden (300 Minuten) Zeit für alle Aufgaben. Die 3 Teilaufgaben sind unabhängig voneinander. Sie können die Reihenfolge und die Zeit frei wählen. Wir empfehlen: höchstens 2 Stunden für Aufgabe 1, 1,5 Stunden für Aufgabe 2 und 1,5 Stunden für Aufgabe 3.

Hilfsmittel

Im Labor steht Visual Studio mit .NET 10 oder IntelliJ Ultimate mit mindestens JDK 21 bereit. Das Projekt liegt auf dem Netzlaufwerk. Sie müssen Ihre Lösung am Ende also nicht extra kopieren. **Schließen** Sie am Ende **alle Programme** und lassen Sie den PC eingeschaltet. Erlaubte Hilfsmittel finden Sie im Ordner **R:\exams**.

Wichtiger Hinweis vor Arbeitsbeginn



Tragen Sie Ihre Daten (Klasse, Name und Accountname) in die Datei *README.md* unter *SPG_Fachtheorie/README.md* ein. Sie sehen die Datei nach dem Öffnen der Solution unter *Solution Items*. **Ohne diese Angaben können wir Ihre Arbeit nicht zuordnen und nicht bewerten!**



Während der Prüfung gibt es keinen Internetzugang. Visual Studio: Verwenden Sie **nie** *Build - Rebuild Solution*. Das löscht die lokalen Pakete. Verwenden Sie immer *Build - Build Solution* (F6).

Teilaufgabe 1: Object Relation Mapping

Das Domänenmodell ist technologie-neutral beschrieben. Es nutzt UML-Namensregeln: Felder beginnen mit einem kleinen Buchstaben und die Datentypen sind neutral. Die neutralen Datentypen entsprechen diesen Typen:

UML (neutral)	.NET (C#)	Java
Integer	int	Integer / int
String	string	String
Decimal	decimal	BigDecimal
Boolean	bool	Boolean / boolean
Date	DateOnly	LocalDate
DateTime	DateTime	LocalDateTime

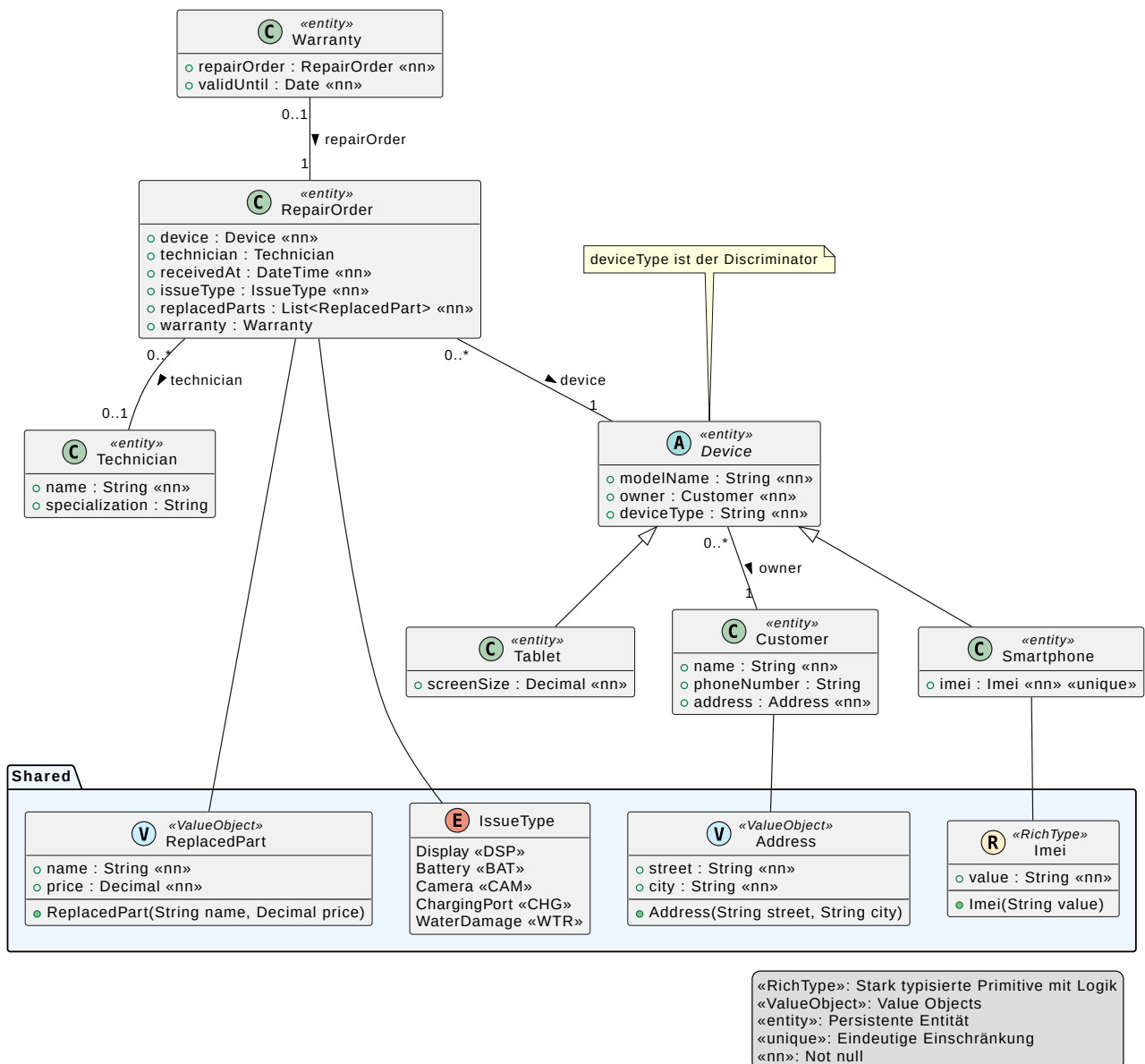
In .NET sind die *nullable reference types* aktiv. Felder ohne `<<nn>>` (not null) sind daher *nullable* (zum Beispiel `string?` in C# oder ein optionales Feld in Java).

Neue Smartphones werden immer teurer. Gleichzeitig verlangt die EU, dass Geräte leichter reparierbar sein müssen. Deshalb lassen viele Menschen ihr Handy oder Tablet reparieren, statt ein neues Gerät zu kaufen. Eine Kette von Reparatur-Shops möchte ihre Aufträge mit einer neuen Software verwalten.

Das System speichert die Kundinnen und Kunden (*Customer*). Jeder *Customer* hat eine Adresse (*Address*). Ein *Customer* besitzt Geräte (*Device*). Es gibt zwei Arten von Geräten: *Smartphone* und *Tablet*. Jedes *Smartphone* hat eine *IMEI*. Das ist eine weltweit eindeutige Nummer mit 15 Ziffern, die das Gerät im Mobilfunknetz identifiziert.

Bringt ein *Customer* ein Gerät in den Shop, entsteht ein Reparaturauftrag (*RepairOrder*). Eine Technikerin oder ein Techniker (*Technician*) führt die Reparatur durch. Beim Anlegen des Auftrags steht der *Technician* oft noch nicht fest. Die getauschten Ersatzteile (*ReplacedPart*) speichert das System als Liste im Auftrag. Nach der Reparatur kann der Shop für einen Auftrag **höchstens eine** Garantie (*Warranty*) ausstellen.

So sieht das Domain Model aus:



Arbeitsauftrag

Erstellung der Modelklassen

Setzen Sie das Diagramm als Modelklassen für den OR Mapper um.

Im Projekt *Aufgabe1* finden Sie leere Klassen und die Klasse *RepairContext* (C#). Nutzen Sie diese.

Beachten Sie dabei folgende Punkte:

Allgemeine Anforderungen

- Legen Sie die nötigen Konstruktoren an. Ein *public* Konstruktor soll alle Properties aus dem

Modell setzen. Fügen Sie bei Bedarf weitere Konstruktoren hinzu, die der OR Mapper braucht.

- Beachten Sie Regeln wie *not null* (<<nn>>).
- Nutzen Sie eigene primary keys mit dem Namen *Id* (autoincrement). Eine Ausnahme gilt nur, wenn im Modell mit <<PK>> ein eigener Schlüssel (PK) angegeben ist.
- Der Name eines Foreign Keys ist: Property-Name + Name des PK (zum Beispiel *OwnerId* für das Property *owner*). Das ist zum Beispiel wichtig, wenn Sie den FK einer dependent Entity selbst setzen.

Fachliche Anforderungen

- Die Tabellennamen sollen in der Einzahl entstehen (*Customer*, *Technician*, *Device*, *RepairOrder*, ...).
- Die Klasse *Device* ist abstrakt. Speichern Sie *Smartphone* und *Tablet* gemeinsam in der Tabelle *Device* (Table per Hierarchy).
- Definieren Sie das Attribut *Device.deviceType* als discriminator.
- Zu einer *RepairOrder* gibt es höchstens eine *Warranty*. Setzen Sie diese 1:0..1 Beziehung um.
- Das Attribut *Smartphone.imei* braucht ein *unique constraint*.
- Definieren Sie das Attribut *Smartphone.imei* als *rich type* und speichern Sie es als *String* (in SQLite als *TEXT*).
- Definieren Sie das Attribut *Customer.address* als *value object*. Die Spalten sollen in der Datenbank *Address_Street* und *Address_City* heißen (das ist das Standardverhalten in EF Core).
- Definieren Sie das Attribut *RepairOrder.replacedParts* als Liste von *value objects*. Die neue Tabelle soll *ReplacedPart* heißen. Sorgen Sie durch die Konfiguration dafür, dass der primary key der neuen Tabelle *Id* heißt.
- Speichern Sie die Enum *RepairOrder.issueType* als String mit diesen Werten: *DSP* für Display, *BAT* für Battery, *CAM* für Camera, *CHG* für ChargingPort und *WTR* für WaterDamage.
- Prüfen Sie in der Klasse *Imei* den Wert im Konstruktor. Der Wert muss **genau 15 Zeichen** lang sein und darf **nur Ziffern** (0 bis 9) enthalten. Ist das nicht der Fall, werfen Sie eine *ArgumentException* (Java: *IllegalArgumentException*).
- Prüfen Sie in der Klasse *ReplacedPart* den Preis im Konstruktor. Ist *price* kleiner als 0, werfen Sie eine *ArgumentException* (Java: *IllegalArgumentException*).



Für die Prüfung der Ziffern können Sie in C# die Methode `char.IsAsciiDigit()` verwenden. In Java hilft `value.matches("[0-9]{15}")`.



Wenn rich types oder enums Probleme machen, können Sie sie auch als einfaches String-Feld umsetzen.

Vorgegebene Tests

Der vorgegebene Test *T00_CanCreateDatabaseTest* in *Aufgabe1.Test/Aufgabe1MasterTests* prüft, ob Ihre Implementierung überhaupt eine Datenbank erzeugen kann. **Das ist die Grundvoraussetzung für Punkte in Aufgabe 1.**

Der vorgegebene Test *T00_SchemaTest* in *Aufgabe1.Test/Aufgabe1MasterTests* prüft das ganze Modell. Er schickt dafür *INSERT*-Befehle für alle Tabellen an die Datenbank ("happy path").

Verfassen von Tests

Schreiben Sie in der Klasse *RepairContextTests* (Projekt *Aufgabe1.Test*) Testmethoden. Diese Tests sollen zeigen, dass Sie den OR Mapper richtig konfiguriert haben.

- *T01_InsertRepairOrderWithWarrantyTest* zeigt: Sie können eine *RepairOrder* für ein *Smartphone* zusammen mit einer *Warranty* speichern.
- *T02_SmartphoneImeiIsUniqueTest* zeigt: Ein zweites *Smartphone* mit derselben IMEI führt beim Speichern zu einer *DbUpdateException* (Java: *DataIntegrityViolationException*).
- *T03_AddRepairOrderWithReplacedPartsTest* zeigt: Sie können eine *RepairOrder* mit mehreren *ReplacedParts* speichern. Leeren Sie nach dem Speichern den *ChangeTracker* (C#: *db.ChangeTracker.Clear()*), damit die Daten wirklich aus der Datenbank gelesen werden. Achten Sie im Assert darauf: Sie laden die *RepairOrder* ohne *Include* und greifen trotzdem auf die Liste der *ReplacedParts* zu (prüfen Sie die Anzahl).
- *T04_IimeiThrowsExceptionWhenInvalidTest* zeigt: Der Konstruktor von *Imei* wirft bei einem ungültigen Wert eine *ArgumentException*. Prüfen Sie einen Wert mit 14 Ziffern und einen Wert mit 15 Zeichen, der einen Buchstaben enthält.

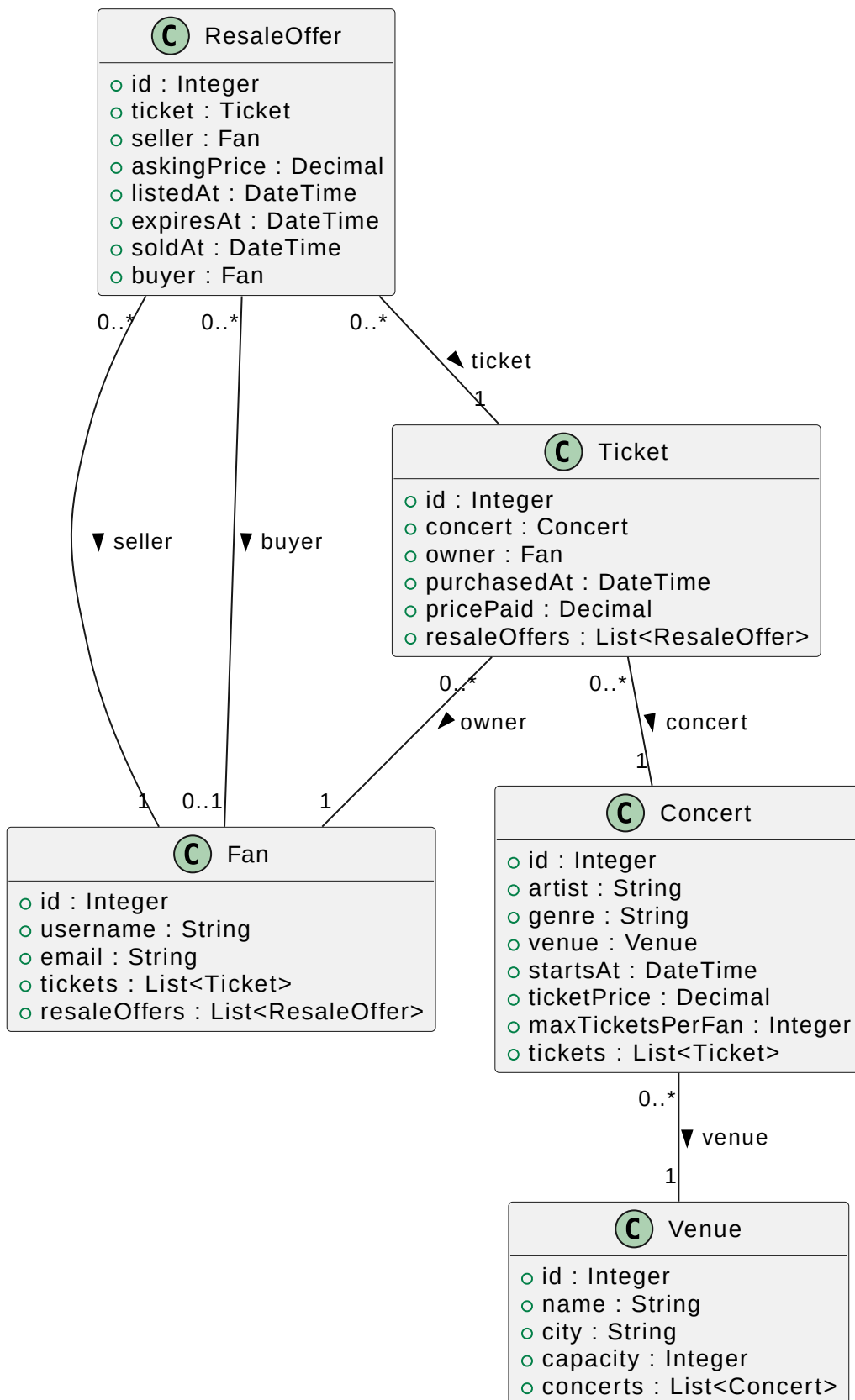
Teilaufgabe 2: Abfragen und Servicemethoden

Wenn eine Band ihre Tour ankündigt, sind die Tickets oft nach wenigen Minuten ausverkauft. Kurz danach tauchen dieselben Tickets auf Resale-Plattformen zu viel höheren Preisen auf. Das folgende Modell beschreibt eine einfache Ticketplattform mit einem fairen Resale-Markt.

Ein *Venue* ist ein Veranstaltungsort (zum Beispiel eine Halle oder ein Stadion) mit einer maximalen Anzahl an Plätzen (*capacity*). In einem *Venue* finden Konzerte (*Concert*) statt. Jedes *Concert* hat einen Ticketpreis (*ticketPrice*) und ein Limit, wie viele Tickets ein Fan höchstens kaufen darf (*maxTicketsPerFan*). Ein *Fan* kauft Tickets (*Ticket*). Das Feld *Ticket.owner* zeigt, wem das Ticket gerade gehört. Das Feld *Ticket.pricePaid* ist der Preis, den der Fan beim Kauf bezahlt hat.

Kann ein *Fan* nicht zum Konzert gehen, bietet er sein Ticket in einem *ResaleOffer* zum Weiterverkauf an. Der *Fan*, der das Angebot erstellt, ist der *seller*. Solange niemand das Ticket gekauft hat, sind *soldAt* und *buyer* *null*. Ein Angebot mit *soldAt* gleich *null* nennen wir **offen**.

Das Domain Model ist technologieneutral beschrieben (UML-Namensregeln).



Fan.resaleOffers enthält nur die Angebote, bei denen der Fan der *seller* ist.

Musterdaten

Im Ordner *datafiles* liegen Musterdaten im TSV-Format. Sie können diese Textdateien in Ihrer IDE ansehen.

Arbeitsauftrag

In der Datei *BaseModel/Infrastructure/TicketContext.cs* ist ein fertiger *DbContext* vorhanden (C#). In Java finden Sie fertige Repositories und Fixtures. Das Modell ist bereits in *BaseModel/Model* umgesetzt. **Sie müssen das Modell nicht selbst implementieren. Sie verwenden es nur in Ihren Servicemethoden.** Schreiben Sie Ihren Code in die Klasse *Aufgabe2/TicketService*. Verwenden Sie in C# den *TicketContext*, den Sie über Dependency Injection bekommen, bzw. die vorgegebenen Repositories in Java.



Die Anzahl der **verkauften Tickets** eines Konzerts ist die Anzahl der *Ticket*-Datensätze dieses Konzerts. Ein Weiterverkauf über ein *ResaleOffer* erzeugt kein neues Ticket. Er ändert nur den *owner* des bestehenden Tickets.

Implementierung der Servicemethoden

Setzen Sie die folgenden Abfragen und Servicemethoden um:

List<VenueRevenue> GetRevenuePerVenue(int year)

Die Methode liefert eine Statistik pro Veranstaltungsort (*Venue*). Geben Sie **alle** Venues zurück, auch Venues ohne Konzerte im Jahr *year*. Berücksichtigen Sie bei den Werten nur Konzerte, die im Jahr *year* stattfinden (Jahr von *Concert.startsAt*). Geben Sie pro *Venue* diese Werte zurück:

- *Name* und *City* des Venues.
- *ConcertCount*: die Anzahl der Konzerte im Jahr *year* in diesem Venue.
- *SoldTickets*: die Anzahl der verkauften Tickets aller dieser Konzerte.
- *Revenue*: die Summe von *Ticket.pricePaid* aller dieser Tickets.

Ein Venue ohne Konzerte oder ohne verkaufte Tickets im Jahr *year* hat bei den entsprechenden Werten den Wert 0. Sortieren Sie das Ergebnis **absteigend** nach *Revenue*. Nutzen Sie diesen Record als Rückgabetypp.

```
// C#
public record VenueRevenue(
    string Name, string City, int ConcertCount, int SoldTickets, decimal Revenue);
```

```
// Java
public record VenueRevenue(
```

```
String name, String city, int concertCount, int soldTickets, BigDecimal revenue) {}
```



Sie summieren hier zweimal. Pro Konzert summieren Sie *Ticket.pricePaid* über alle Tickets. Danach summieren Sie diese Konzert-Summen über alle Konzerte des Venues im Jahr *year*.

List<ConcertAvailability> GetAvailableConcerts(string city, DateTime from)

Die Methode sucht Konzerte, für die man noch Tickets kaufen kann. Geben Sie alle Konzerte zurück, für die **alle** Bedingungen gelten:

- Das *Venue* liegt in der Stadt *city* (*Venue.city* ist gleich *city*).
- Das Konzert beginnt zum Zeitpunkt *from* oder später (*startsAt* $\geq$ *from*).
- Das Konzert ist nicht ausverkauft (die Anzahl der verkauften Tickets ist kleiner als *Venue.capacity*).

Geben Sie pro Konzert auch die Anzahl der freien Plätze (*FreeSeats*) zurück. Das ist *Venue.capacity* minus die Anzahl der verkauften Tickets. Sortieren Sie das Ergebnis **aufsteigend** nach *startAt*. Nutzen Sie diesen Record als Rückgabetypp.

```
// C#
public record ConcertAvailability(
    int ConcertId, string Artist, string VenueName,
    DateTime StartsAt, int FreeSeats);
```

```
// Java
public record ConcertAvailability(
    int concertId, String artist, String venueName,
    LocalDateTime startsAt, int freeSeats) {}
```

List<Fan> GetScalpers(decimal maxMarkupPercent)

Ein *Scalper* ist eine Person, die Tickets mit Gewinn weiterverkaufen möchte. Geben Sie alle *Fans* zurück, die **mindestens ein** *ResaleOffer* als *seller* erstellt haben, bei dem der Preis zu hoch ist. Der Preis ist zu hoch, wenn gilt: $\text{askingPrice} > \text{pricePaid} * (100 + \text{maxMarkupPercent}) / 100$. *pricePaid* ist dabei der Wert aus dem Ticket des Angebots. Berücksichtigen Sie offene und schon verkaufte Angebote.



Beispiel: *maxMarkupPercent* ist 20 und *pricePaid* ist 80 Euro. Die Grenze ist also $80 * 120 / 100 = 96$ Euro. Ein Angebot mit 100 Euro ist zu hoch, ein Angebot mit 96 Euro nicht.

Ticket BuyTicket(int fanId, int concertId, DateTime purchasedAt)

Diese Methode verkauft ein neues Ticket für ein Konzert. Prüfen Sie vorher diese Bedingungen in dieser Reihenfolge:

- Gibt es keinen *Fan* mit der *fanId* in der Datenbank, werfen Sie eine *TicketException* mit dem Text "*Invalid fan.*".
- Gibt es kein *Concert* mit der *concertId* in der Datenbank, werfen Sie eine *TicketException* mit dem Text "*Invalid concert.*".
- Ist *purchasedAt* gleich oder später als *Concert.startsAt*, werfen Sie eine *TicketException* mit dem Text "*Concert has already started.*".
- Ist die Anzahl der verkauften Tickets für dieses Konzert schon gleich *Venue.capacity* (oder größer), werfen Sie eine *TicketException* mit dem Text "*Concert is sold out.*".
- Besitzt der *Fan* für dieses Konzert schon so viele Tickets wie *Concert.maxTicketsPerFan* (oder mehr), werfen Sie eine *TicketException* mit dem Text "*Ticket limit reached.*". Zählen Sie dafür die Tickets, bei denen der Fan der *owner* ist.

Sind alle Bedingungen erfüllt, legen Sie ein neues *Ticket* an und speichern Sie es in der Datenbank. Der *owner* ist der Fan, *purchasedAt* ist der übergebene Zeitpunkt. Setzen Sie *pricePaid* auf den aktuellen *ticketPrice* des Konzerts. Geben Sie das neue Ticket zurück.

Testen Ihrer Implementierung

In *Aufgabe2.Test/Aufgabe2MasterTests* gibt es schon fertige Tests für Ihre Implementierung. Sie müssen hier keine eigenen Unittests schreiben.

Teilaufgabe 3a: Erstellen einer RESTful API

Im Projekt *Aufgabe3* gibt es die Klasse *ResaleOffersController*. Diese Klasse bekommt über Dependency Injection ein fertiges Service (*ResaleService*). **Verwenden Sie dieses fertige Service. Sie brauchen Ihr Service aus Aufgabe 2 nicht.** Das Modell ist dasselbe wie in Aufgabe 2.

Das Service verwendet einen *TimeProvider* (C#) bzw. eine *Clock* (Java), um die aktuelle Zeit zu bestimmen. Beide sind in *Program.cs* bzw. in der Konfiguration schon registriert.

Das Service hat 3 Methoden bzw. Properties:

IQueryable<ResaleOffer> ResaleOffers

Liefert alle *ResaleOffer*-Datensätze aus der Datenbank. Sie brauchen das für die GET-Methoden im Controller.

Task<ResaleOffer> AddOffer(AddResaleOfferCmd cmd)

Legt aus dem Command ein neues *ResaleOffer* an und gibt es zurück. *listedAt* ist die aktuelle Zeit. Wirft eine *ResaleServiceException*, wenn das Angebot nicht möglich ist. Das ist der Fall, wenn es das Ticket nicht gibt, wenn der *seller* nicht der *owner* des Tickets ist, wenn es für das Ticket schon ein offenes Angebot gibt, wenn das Konzert schon begonnen hat oder wenn *expiresAt* nach dem Beginn des Konzerts liegt.

Task PurchaseOffer(int id, int buyerId)

Verkauft das Ticket aus dem Angebot an den Fan mit der ID *buyerId*. Das Service setzt *soldAt* auf die aktuelle Zeit, setzt *buyer* und ändert den *owner* des Tickets. Wirft eine *ResaleOfferNotFoundException*, wenn es kein Angebot mit dieser *id* gibt. Wirft eine *ResaleServiceException*, wenn der Kauf nicht möglich ist. Das ist der Fall, wenn das Angebot schon verkauft ist, wenn es den *buyer* nicht gibt oder wenn *buyer* und *seller* dieselbe Person sind.

Zu implementierende Endpunkte

GET /resaleOffers?artist=(string)&maxPrice=(decimal)

Diese Route gibt eine Liste aller **offenen** Angebote (*soldAt* ist *null*) zurück. Beide Query-Parameter sind **optional**:

- Ist *artist* angegeben, liefern Sie nur Angebote für Konzerte, bei denen *Concert.artist* genau gleich *artist* ist.
- Ist *maxPrice* angegeben, liefern Sie nur Angebote mit *askingPrice* kleiner oder gleich *maxPrice*.
- Sind beide Parameter angegeben, müssen beide Bedingungen gelten.

Sortieren Sie die Liste aufsteigend nach dem Beginn des Konzerts (*Concert.startsAt*). Nutzen Sie diesen Record für die Rückgabe:

```
// C#
public record ResaleOfferDto(
    int Id, string Artist, DateTime ConcertStartsAt, string VenueName,
    decimal OriginalPrice, decimal AskingPrice, string SellerUsername);
```

```
// Java
public record ResaleOfferDto(
    int id, String artist, LocalDateTime concertStartsAt, String venueName,
    BigDecimal originalPrice, BigDecimal askingPrice, String sellerUsername) {}
```

OriginalPrice ist der Wert von *Ticket.pricePaid*.

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Ein Array von <i>ResaleOfferDto</i> . Es enthält alle offenen Angebote, die zu den Filtern passen (eventuell ein leeres Array).

GET /resaleOffers/{id}

Diese Route gibt das *ResaleOffer* mit der übergebenen ID zurück. Sie liefert auch schon verkaufte Angebote.

Nutzen Sie diesen Record für die Rückgabe:

```
// C#
public record ResaleOfferDetailDto(
    int Id, int TicketId, string Artist, decimal AskingPrice,
    DateTime ListedAt, DateTime ExpiresAt,
    DateTime? SoldAt, string? BuyerUsername);
```

```
// Java
public record ResaleOfferDetailDto(
    int id, int ticketId, String artist, BigDecimal askingPrice,
    LocalDateTime listedAt, LocalDateTime expiresAt,
    LocalDateTime soldAt, String buyerUsername) {}
```

Ist das Angebot noch offen, sind *SoldAt* und *BuyerUsername* *null*.

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Ein (1) <i>ResaleOfferDetailDto</i> Objekt mit den entsprechenden Daten.
404	Die ID wurde nicht gefunden.

POST /resaleOffers

Diese Route legt einen neuen *ResaleOffer*-Datensatz an. Nutzen Sie dieses Command-Objekt:

```
// C#  
public record AddResaleOfferCmd(  
    int TicketId, int SellerId, decimal AskingPrice, DateTime ExpiresAt);
```

```
// Java  
public record AddResaleOfferCmd(  
    int ticketId, int sellerId, BigDecimal askingPrice, LocalDateTime expiresAt) {}
```

Beachten Sie bei der Validierung diese Regeln:

- *TicketId* ist mindestens 1.
- *SellerId* ist mindestens 1.
- *AskingPrice* liegt zwischen 1 und 10000 (inklusive).

Bauen Sie zusätzlich eine Validierung ein, die die aktuelle Zeit braucht. Nutzen Sie dafür *IValidatableObject* (C#) bzw. eine klassenweite Validierung (Java). Es gilt diese Regel:

- *ExpiresAt* muss **nach** der aktuellen Zeit liegen. Ein Angebot kann nicht schon in der Vergangenheit ablaufen (Fehlertext zum Beispiel *Expiry date must be in the future.*).



Verwenden Sie für die aktuelle Zeit **nicht** `DateTime.Now` bzw. `LocalDateTime.now()`, sondern den registrierten *TimeProvider* bzw. die *Clock*. Die automatischen Tests setzen die Zeit auf einen festen Wert. In C# bekommen Sie den *TimeProvider* in der Methode `Validate()` mit `validationContext.GetRequiredService<TimeProvider>()`. Die aktuelle Zeit liefert dann `GetLocalNow().DateTime`. In Java können Sie die *Clock* in Ihren *ConstraintValidator* injizieren und `LocalDateTime.now(clock)` verwenden.

Bei einem Validierungsfehler oder einem Fehler der Business-Logik (*ResaleServiceException*) senden Sie HTTP 400 (bad request) mit einem RFC-Problem-Detail-Body.

War das Einfügen erfolgreich, senden Sie HTTP 201 Created. Senden Sie im Payload die ID des neuen *ResaleOffer*-Objekts im Property *id*. Senden Sie im Location-Header die Adresse der passenden GET-Methode (`GET /resaleOffers/{id}`).

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
201	Das Angebot konnte angelegt werden.

HTTP Status	Bedingung
400	Ein Validierungs- oder Servicefehler ist aufgetreten (z. B. eine nicht vorhandene <i>TicketId</i> oder ein <i>ExpiresAt</i> in der Vergangenheit).

PATCH /resaleOffers/{id}/purchase

Mit dieser Route kauft ein Fan das Ticket aus einem offenen Angebot. Die ID des *ResaleOffer*-Datensatzes bekommen Sie über den Routing-Parameter. Die ID des kaufenden Fans steht im Request-Body. Nutzen Sie dieses Command-Objekt:

```
// C#
public record PurchaseResaleOfferCmd(int BuyerId);
```

```
// Java
public record PurchaseResaleOfferCmd(int buyerId) {}
```

BuyerId muss mindestens 1 sein. Rufen Sie danach die Methode *PurchaseOffer* des Service auf.

Gibt es kein *ResaleOffer* mit dieser ID, senden Sie HTTP 404 (not found) mit einem RFC-Problem-Detail-Body. Bei einem Validierungsfehler oder einer *ResaleServiceException* senden Sie HTTP 400 (bad request) mit einem RFC-Problem-Detail-Body.

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
204	Das Ticket wurde erfolgreich gekauft.
400	Ein Validierungs- oder Servicefehler ist aufgetreten (z. B. das Angebot ist schon verkauft).
404	Die ID wurde nicht gefunden.

HTTP-File zum Testen Ihrer Implementierung

Im Projekt *Aufgabe3* gibt es die Datei *Aufgabe3.http*. Damit können Sie Ihre Implementierung von Hand testen.

Teilaufgabe 3b: Erstellen von End-to-End (Integration) Tests (6 Punkte)

Schreiben Sie im Projekt *Aufgabe3.Test* in der Klasse *ResaleOffersControllerTests* Integration-Tests für den Endpunkt *PATCH /resaleOffers/{id}/purchase*. **Testen Sie keine anderen Endpunkte.**

Schreiben Sie Tests für diese zwei Szenarien:

- *PATCH /resaleOffers/{id}/purchase* verkauft ein offenes Angebot und liefert HTTP 204 No Content. Prüfen Sie den Statuscode. Prüfen Sie außerdem in der Datenbank, ob *soldAt* und *buyer* gesetzt sind und ob der *owner* des Tickets jetzt der *buyer* ist.
- *PATCH /resaleOffers/{id}/purchase* kann ein Angebot nicht verkaufen, weil es schon verkauft ist. Prüfen Sie den Statuscode.



Suchen Sie die passenden Angebote (offen bzw. schon verkauft) im Test über die Datenbank. Verlassen Sie sich nicht auf fixe IDs aus den Musterdaten.